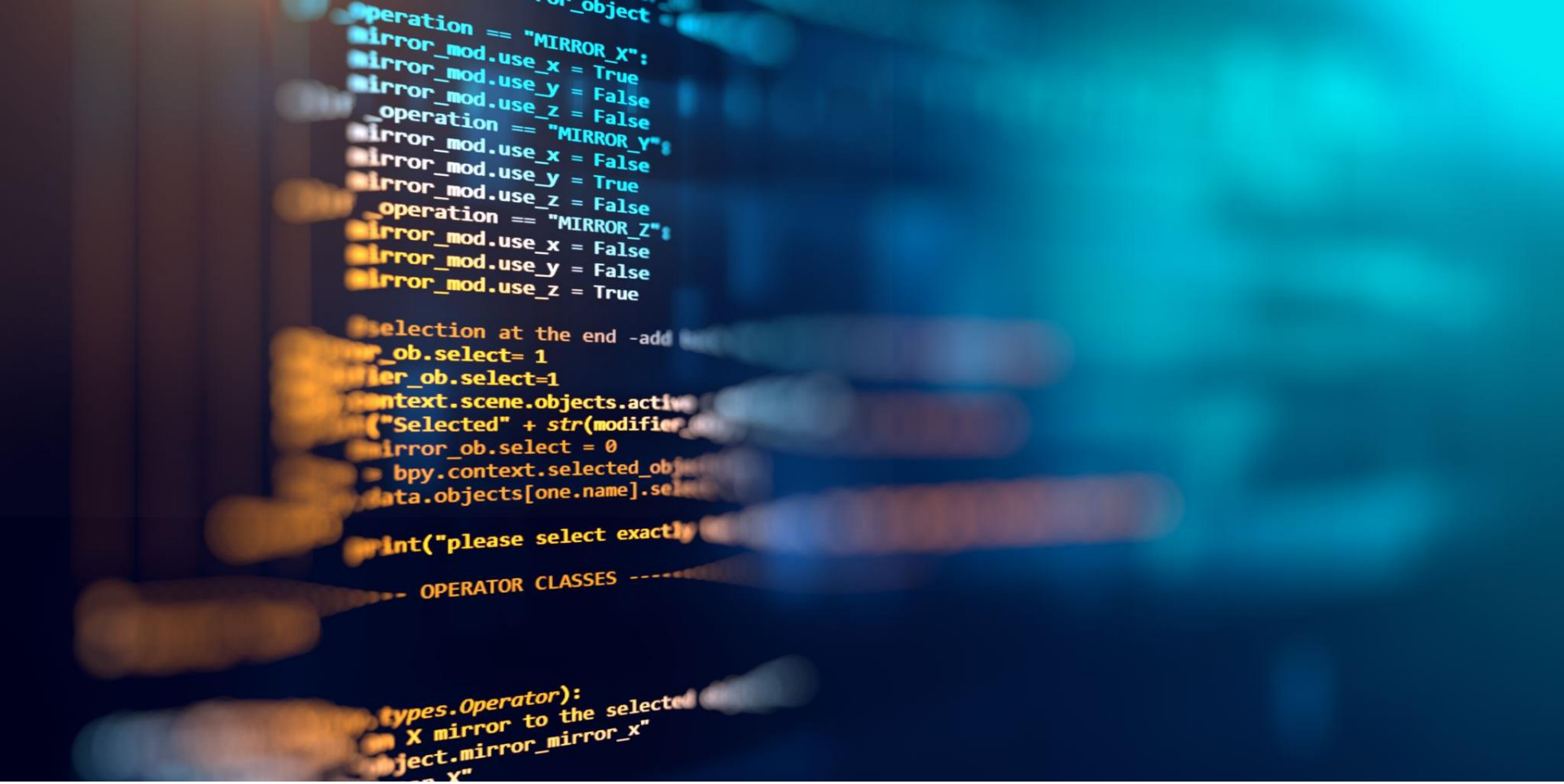


MARKOV DECISION PROBLEM, BANDITS, AND DYNAMIC PROGRAMMING

An Overview with Explanations

MARKOV DECISION PROCESS

The Markov Decision Process (MDP) is a fundamental concept in reinforcement learning that provides a mathematical framework for modeling decision-making problems where outcomes are partly random and partly under the control of an agent.

MARKOV DECISION PROBLEM (MDP)

Key Components:

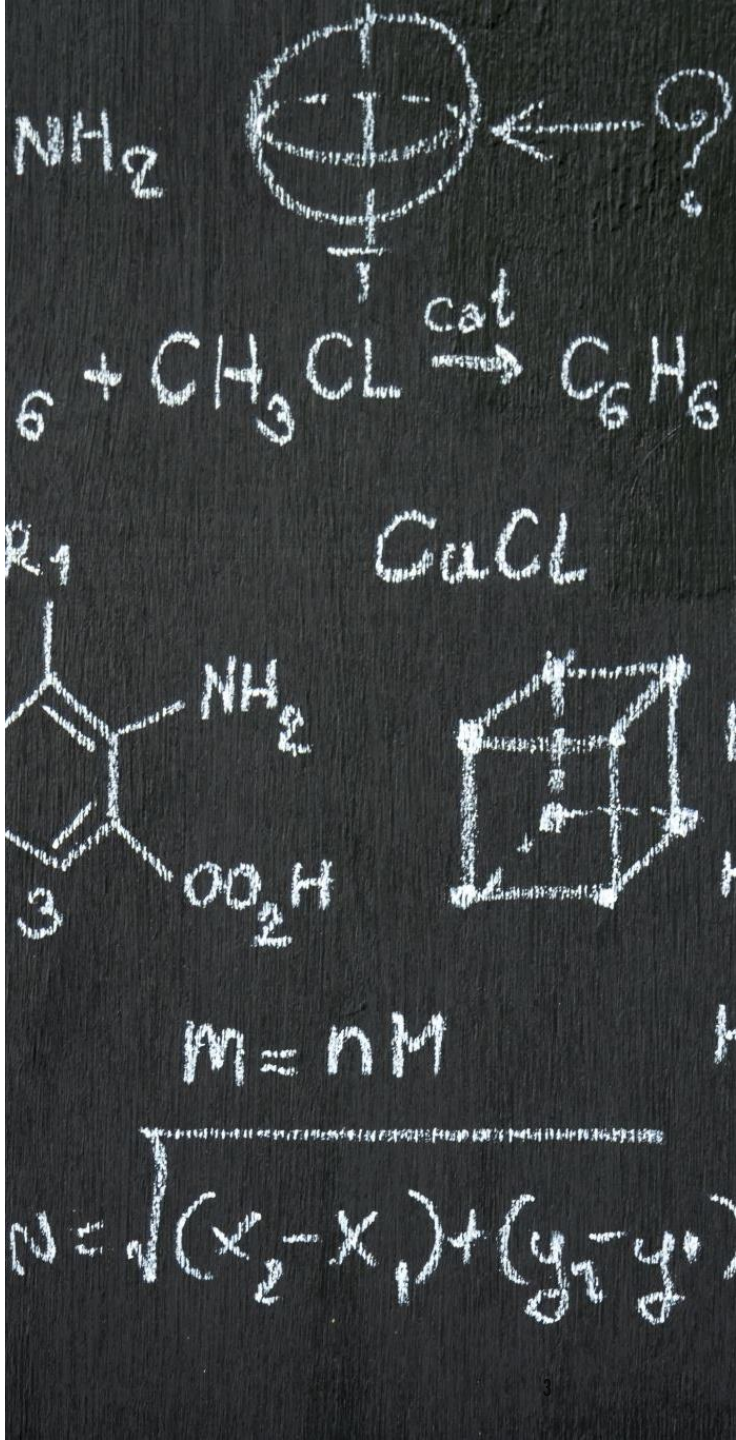
- States (S): A finite set of states representing all possible situations.

- Actions (A): A finite set of actions available to the agent.

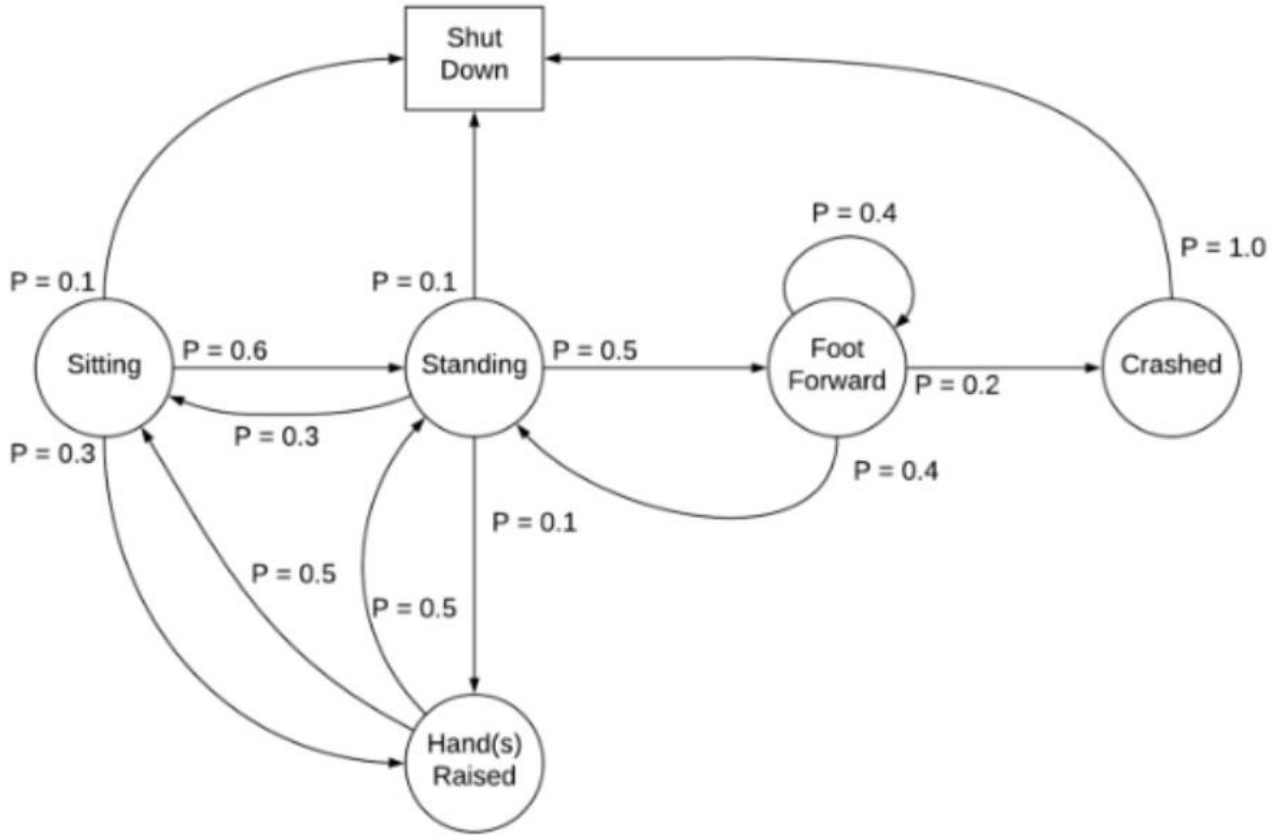
- Transition Probabilities (P): Probability of transitioning from one state to another given an action.

- Rewards (R): Immediate reward received after transitioning from one state to another.

- Policy (π): A strategy defining the agent's behavior by specifying actions in each state.



TRANSITION PROBABILITIES



https://builtin.com/machine-learning/markov-decision-process

MDP COMPONENTS

An MDP is defined by a tuple $(S,A,P,R,\gamma)$where:

- **S (State Space):** A finite or infinite set of states representing the environment.
- **A (Action Space):** A set of actions available to the agent in each state.
- **P (Transition Probability):** A probability function **P(s' | s, a)** that defines the likelihood of transitioning from state **s** to **s'** after taking action **a**.
- **R (Reward Function):** A function **R(s, a, s')** that assigns a reward for moving from state **s** to **s'** via action **a**.
- **γ (Discount Factor):** A value in the range **[0,1]** that determines the importance of future rewards.

MARKOV PROPERTY

MDP follows the **Markov Property:** The state contains all the relevant information from the past to predict future.

So, if I say that the state **S<t>** is Markov, that means, it has all the important representations of the environment from previous states(which means you can throw away all the previous states). Think of it in this way: when you have the boarding pass, you do not need your ticket anymore to board the plane; your boarding pass already contains all the necessary information about boarding.

https://medium.com/@bibekchaudhary/markov-decision-process-mdp-simplified-1ae44cf53cc1

A state S_t is *Markov* if and only if

$$\mathbb{P}[S_{t+1} \mid S_t] = \mathbb{P}[S_{t+1} \mid S_1, \ldots, S_t]$$

pic taken from David Silver lecture slide

REWARD

$R(s,a,s')$

where:

- s is the current state.
- a is the action taken.
- s' is the resulting state after taking action aa.

In practice, the reward function can vary depending on the specific problem and environment. For example:

- In a deterministic setting, R(s,a,s') might be a fixed value.
- In a stochastic setting, R(s,a,s') might be drawn from a probability distribution.

EXAMPLE

$$R(s, a, s') = \begin{cases} 10 & \text{if } s' \text{ is the goal state} \\ -5 & \text{if } s' \text{ is an obstacle state} \\ -1 & \text{otherwise} \end{cases}$$

EXAMPLE-DETERMINISTIC REWARD

Reward Function:

- **Goal State**: +10
- **Obstacle State**: -5
- **Other States**: -1

EXAMPLE-STOCHASTIC REWARD

- **Goal State**: +10 with probability 0.8, +5 with probability 0.2
- **Obstacle State**: -5 with probability 0.9, -10 with probability 0.1
- **Other States**: -1 with probability 0.7, -2 with probability 0.3

POLICY

A **policy** defines the agent's strategy for selecting actions in each state. It can be:

- **Deterministic ($\pi(s)$)**: Always selects a fixed action for a given state.

- **Stochastic ($\pi(a|s)$)**: Assigns probabilities to different actions for a given state.

EXAMPLE-DETERMINISTIC POLICY

Policy Table:

State	Action
(0,0)	Right
(0,1)	Right
(0,2)	Down
(1,0)	Down
(1,1)	Left
(1,2)	Down
(2,0)	Right
(2,1)	Right
(2,2)	Goal

STOCHASTIC POLICY

Policy Table:

State	Action Probabilities
(0,0)	Right: 0.7, Down: 0.3
(0,1)	Right: 0.8, Down: 0.2
(0,2)	Down: 0.9, Left: 0.1
(1,0)	Down: 0.6, Right: 0.4
(1,1)	Left: 0.5, Up: 0.5
(1,2)	Down: 0.7, Left: 0.3
(2,0)	Right: 0.9, Up: 0.1
(2,1)	Right: 0.8, Up: 0.2
(2,2)	Goal: 1.0

VALUE FUNCTION

In a Markov Decision Process (MDP), the **value function** is a key concept that helps quantify the long-term benefit of being in a particular state or taking a specific action.

PURPOSE

- **Purpose**: Quantifies the long-term benefit of being in a particular state or taking a specific action.
- **Types**:
  - **State Value Function** (V(s)): Expected cumulative reward starting from state *s*.
  - **Action Value Function** (Q(s,a)): Expected cumulative reward starting from state *s* and taking action *a*.

EXAMPLE

Gridworld Environment

- **Grid**: 3x3 grid.
- **Start State**: (0,0).
- **Goal State**: (2,2).
- **Obstacle**: (1,1).

State Value Function (V(s)

- *V*((0,0))=5
- V((1,0))=6
- V((2,2))= (goal state)

Action Value Function (Q(s,a)

- *Q*((0,0),right)=4
- Q((0,0),down)=5
- Q((1,0),right)=7
- Q((1,0),down)=6
- Q((2,1),right)=10 (moving to the goal state)

- **State Value Function (V(s))**: Better for evaluating states and simpler environments.
- **Action Value Function (Q(s,a))**: Better for detailed action evaluation and complex environments.

STATE VALUE VS ACTION VALUE

$$V(s) = \max_a \left[R(s, a, s') + \gamma V(s') \right]$$

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R(s, a, s') + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

OPTIMAL VALUE FUNCTION

The **optimal value** function, denoted as $Q*(s,a)$, represents the maximum expected cumulative reward that can be obtained by taking action *a* in state *s* and then following the optimal policy thereafter.

Update the policy by choosing actions that maximize the value function.

SOLVING MDPS IN REINFORCEMENT LEARNING

Several algorithms have been developed to solve MDPs within the RL framework. Here are a few key approaches:

Dynamic programming methods, such as Value Iteration and Policy Iteration, are used to solve MDPs when the model of the environment (transition probabilities and rewards) is known.

- **Value Iteration**: Iteratively updates the value function until it converges to the optimal value function.

- **Policy Iteration**: Alternates between policy evaluation and policy improvement until the policy converges to the optimal policy.

EXAMPLE: GRIDWORLD ENVIRONMENT

- **Grid**: 3x3 grid.
- **Start State**: (0,0).
- **Goal State**: (2,2).
- **Obstacle**: (1,1).
- **Rewards**:
  - Goal State: +10
  - Obstacle State: -5
  - Other States: -1
- **Discount Factor**: γ=0.9 **learning rate:** 0.1

GRID

START STATE- (0,0) -1 V(0) 	(0,1) -1 V(0)	(0,2) -1 V(0)
(1,0) -1 V(0)	OBSTACLE (1,1) -5 V(0) 	(1,2) -1 V(0)
(2,0) -1 V(0)	(2,1) -1 V(0)	GOAL (2,2) +10 V(0) 

POLICY

Policy Table:

State	Action
(0,0)	Right
(0,1)	Right
(0,2)	Down
(1,0)	Down
(1,1)	Left
(1,2)	Down
(2,0)	Right
(2,1)	Right
(2,2)	Goal

You can start with an arbitrary policy like this. Using a deterministic policy.

APPROACH-POLICY ITERATION

Policy Iteration

1. **Initial Policy**: Start with an arbitrary policy.
2. **Policy Evaluation**: Compute the value function V(s) for the current policy.
3. **Policy Improvement**: Update the policy by choosing actions that maximize V(s).
4. **Repeat**: Iterate until the policy converges to the optimal policy.

STATE VALUE FUNCTION (V(S))

Calculation Steps:

1. **Initialize**: Start with arbitrary values for V(s) (e.g., all zeros).
2. **Update**: Use the Bellman equation to update V(s) iteratively until convergence.

Bellman Equation:

$$V(s) = \max_a \left[R(s, a, s') + \gamma V(s') \right]$$

- **State**: (0,0)
- **Possible Actions**: "right" to (0,1)
- **Rewards**: -1
- **Next State Values: (Only one for now)**

V(0,0)=max[-1+0.9* 0]=-1

TASK

Update all the value function using the initial policy.

ACTION VALUE FUNCTION

Step-by-Step Calculation

1. Initialize the Action Value Function (Q(s,a))

Start with arbitrary values for Q(s,a). For simplicity, initialize all values to 0.

2. Update the Action Value Function

Use the Q-learning update rule to iteratively update Q(s,a):

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[R(s,a,s') + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

- **State**: (0,0)
- **Possible Actions**: "right" to (0,1)
- **Rewards**: -1
- **Next State Values:**

V(0,0)=0.1[-1+0.9*0-0]= 0.1*-1=-0.1

OPTIMAL POLICY Π(S)

After computing the value function choose the action that maximizes the action value function.

$$\pi(s) = \arg\max Q(s, a)$$

For state (0,0):

- Q((0,0),right)=0.26
- Q((0,0),down)=0
- The policy would choose "right" since Q((0,0),right) is higher.

APPLICATIONS OF MDP

1. **Routing Problems**: MDPs are used to optimize routing decisions in logistics and transportation, ensuring efficient delivery of goods and services.
2. **Maintenance and Repair**: They help in managing the maintenance and repair schedules of dynamic systems, such as machinery and infrastructure, to minimize downtime and costs.
3. **Healthcare**: MDPs assist in determining the optimal number of patients to admit to a hospital, balancing resource availability and patient care quality.
4. **Traffic Management**: They are used to manage wait times at traffic intersections, optimizing traffic flow and reducing congestion1
5. **Robotics**: In designing intelligent machines, MDPs help in decision-making processes to perform tasks autonomously.
6. **Game Design**: MDPs are applied in designing quiz games and other interactive applications to enhance user experience and engagement.

ARTICLE REVIEW

https://d2l.ai/chapter_reinforcement-learning/mdp.html

https://www.geeksforgeeks.org/what-is-markov-decision-process-mdp-and-its-relevance-to-reinforcement-learning/

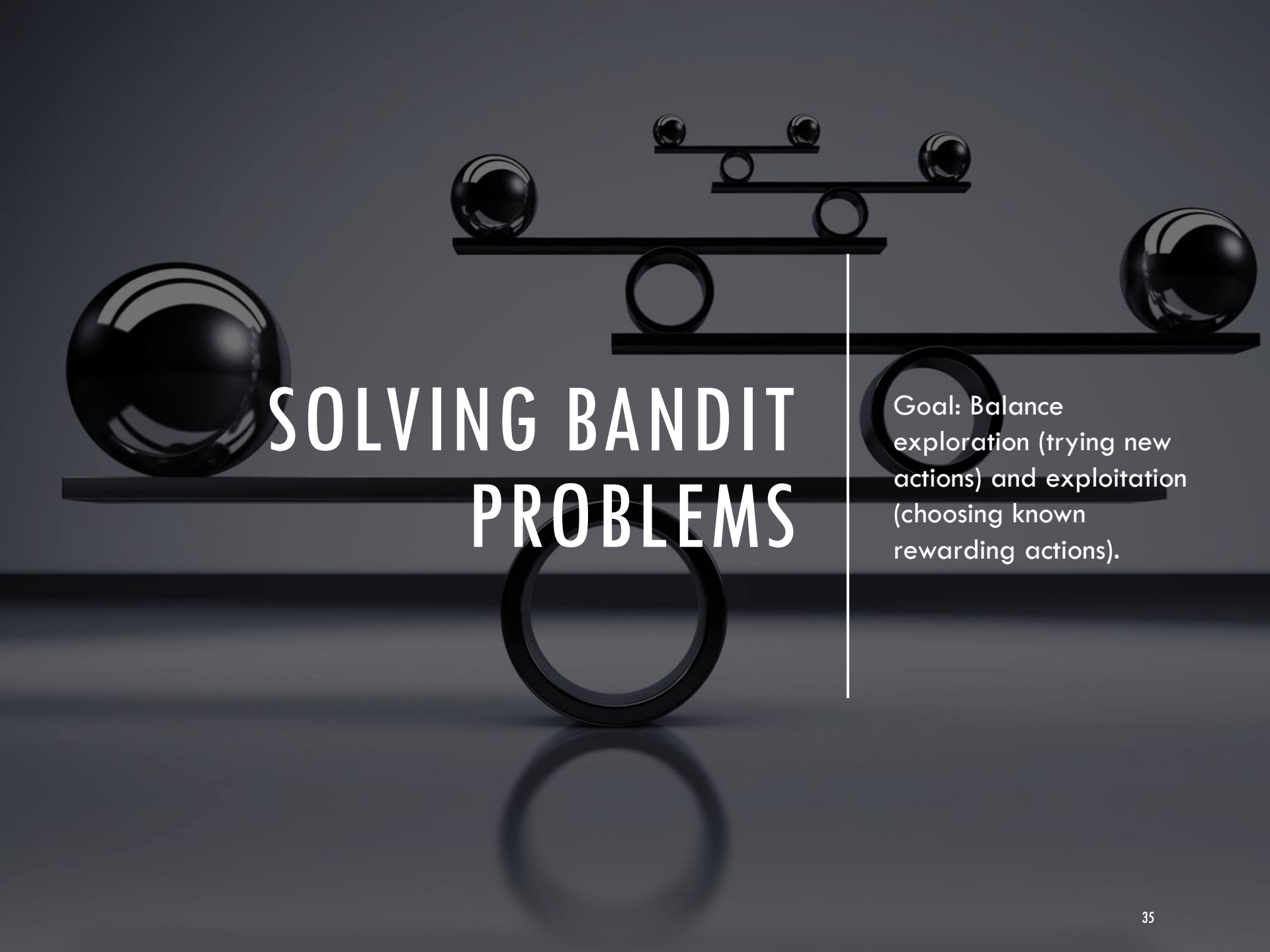
BANDITS

Definition: A simplified reinforcement learning problem where an agent chooses from a set of actions (arms) to maximize cumulative reward.

Types:

- Multi-Armed Bandit: Each action(arm) provides a random reward from a probability distribution.

- Contextual Bandit: The agent observes context(state) before choosing an action (arm), and rewards depend on both the action and context.



SOLVING BANDIT PROBLEMS

Goal: Balance exploration (trying new actions) and exploitation (choosing known rewarding actions).

Methods:

- ε-Greedy: Choose a random action with probability ε, otherwise choose the best-known action.

- Upper Confidence Bound (UCB): Choose actions based on upper confidence bounds of estimated rewards.

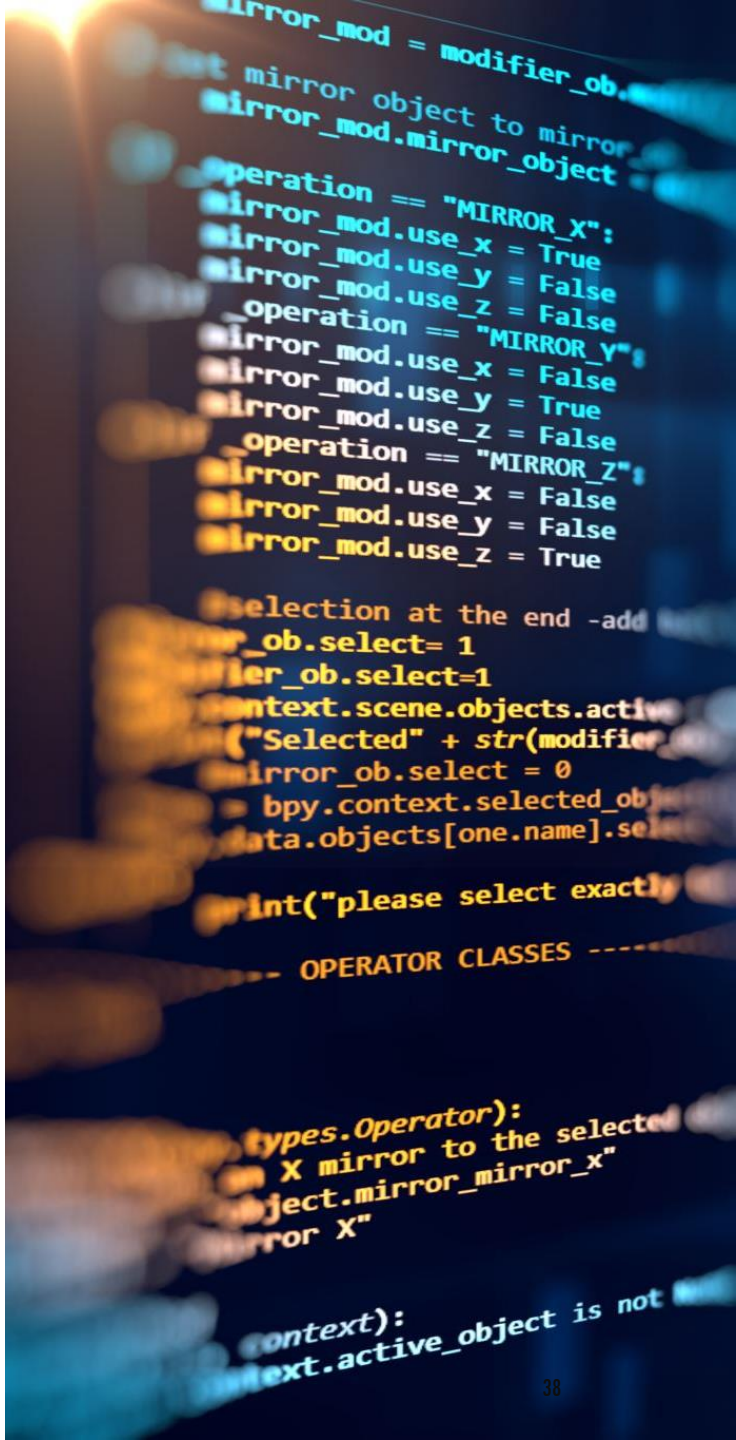
- Thompson Sampling: Choose actions based on probability distributions of estimated rewards.

APPLICATIONS OF BANDIT ALGORITHMS

1. **Online Advertising**: Bandit algorithms are used to optimize ad placements by balancing the exploration of new ads with the exploitation of known successful ads to maximize click-through rates and revenue
2. **Personalized Recommendations**: In e-commerce and streaming services, bandit algorithms help in recommending products, movies, or music by learning user preferences over time and adapting recommendations accordingly
3. **Clinical Trials**: Bandit algorithms facilitate dynamic resource allocation in clinical trials, ensuring that more patients receive potentially effective treatments while still exploring new treatment options
4. **A/B Testing**: They are used to dynamically allocate traffic between different versions of a webpage or feature, optimizing for the best user experience and conversion rates
5. **Dynamic Pricing**: In retail and airline industries, bandit algorithms help in setting prices dynamically based on demand and competition, balancing short-term profits with long-term customer satisfaction
6. **Robotics**: Bandit algorithms are used in robotic control systems to optimize actions and improve performance in uncertain environments

DYNAMIC PROGRAMMING

Definition: A method for solving complex problems by breaking them down into simpler subproblems and solving each subproblem only once.



Key Concepts:

- Overlapping Subproblems: The problem can be broken down into subproblems that are reused multiple times.

- Optimal Substructure: The optimal solution to the problem can be constructed from optimal solutions to its subproblems.

APPLICATIONS OF DYNAMIC PROGRAMMING

Examples:

- Fibonacci Sequence: Computing Fibonacci numbers efficiently using memorization.

- Shortest Path: Finding the shortest path in a graph using algorithms like Bellman-Ford.

FIBONACCI SERIES DEFINITION

$$F(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

EXAMPLE

$4!=4*3!=4*3*2!=4*3*2*1$

USES OF DP

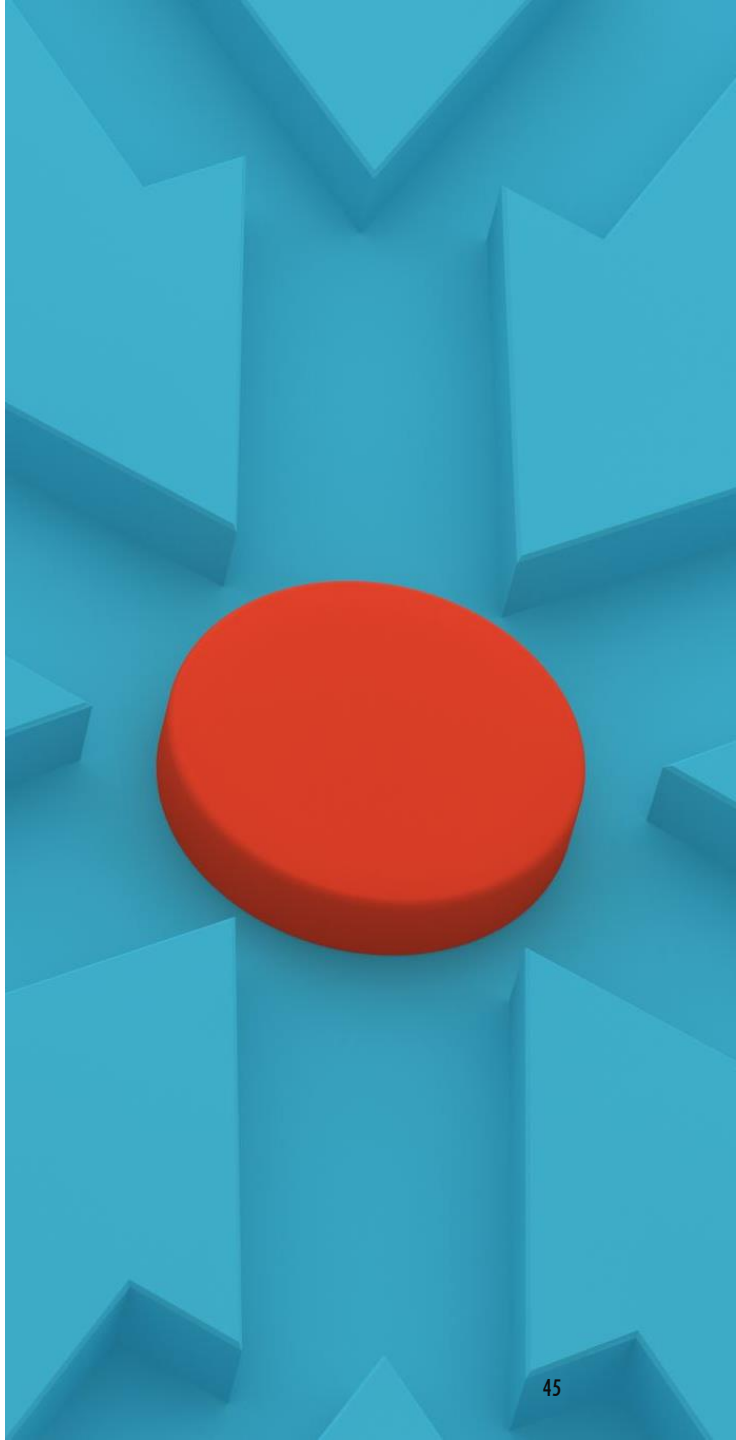
Dynamic programming in RL provides efficient algorithms for solving MDPs by leveraging the Bellman equations and iterative updates. It helps find optimal policies and value functions, making it a powerful tool in reinforcement learning.

TEMPORAL DIFFERENCE LEARNING

- **Definition:** Temporal Difference (TD) learning is a model-free reinforcement learning method that learns by bootstrapping from the current estimate of the value function.

KEY CONCEPTS:

- **Bootstrapping:** refers to the process of updating estimates based on other estimates.
- **TD Target:** The update rule uses the difference between the predicted reward and the observed reward.



TD UPDATE RULE

TD Update Rule

State Value Function ($V(s)$) Update:

$$V(s) \leftarrow V(s) + \alpha \left[R(s, a, s') + \gamma V(s') - V(s) \right]$$

The key formula for TD learning is the TD update rule, which updates the value function based on the difference (or "temporal difference") between consecutive value estimates.

APPLICATIONS OF TD

Temporal Difference (TD) Learning has a wide range of applications across various fields. Here are some notable examples:

1. **Game AI**: TD Learning has been used to develop intelligent agents for games like chess, backgammon, and Go. Notably, it was a key component in the success of TD-Gammon, a backgammon-playing program that achieved expert-level performance
2. **Robotics**: In robotics, TD Learning helps in training robots to perform tasks autonomously by learning from their interactions with the environment. This includes navigation, manipulation, and other complex behavior.
3. **Finance**: TD Learning is applied in financial modeling to predict stock prices and optimize trading strategies. It helps in making decisions based on the temporal patterns observed in financial data
4. **Healthcare**: TD Learning is used in personalized treatment planning, where it helps in optimizing treatment strategies for patients by learning from historical data and patient responses.
5. **Neuroscience**: TD Learning models are used to understand the learning and decision-making processes in the brain, particularly in studying the role of dopamine neurons in reward-based learning
6. **Traffic Management**: TD Learning is applied in optimizing traffic signal timings to reduce congestion and improve traffic flow in urban areas.

POLICY GRADIENT METHODS

- **Definition:** Policy Gradient methods are a family of algorithms that optimize the policy directly by performing gradient ascent on expected rewards
- **Gradient ascent** is like climbing a hill. Imagine you're trying to find the highest point on a hill. You take steps in the direction that goes up the steepest.
- In RL, the "hill" represents the performance of the policy, and the "steps" are adjustments to the policy.
- By following the gradient ascent method, the agent makes changes to its policy to improve its performance, aiming to maximize rewards.

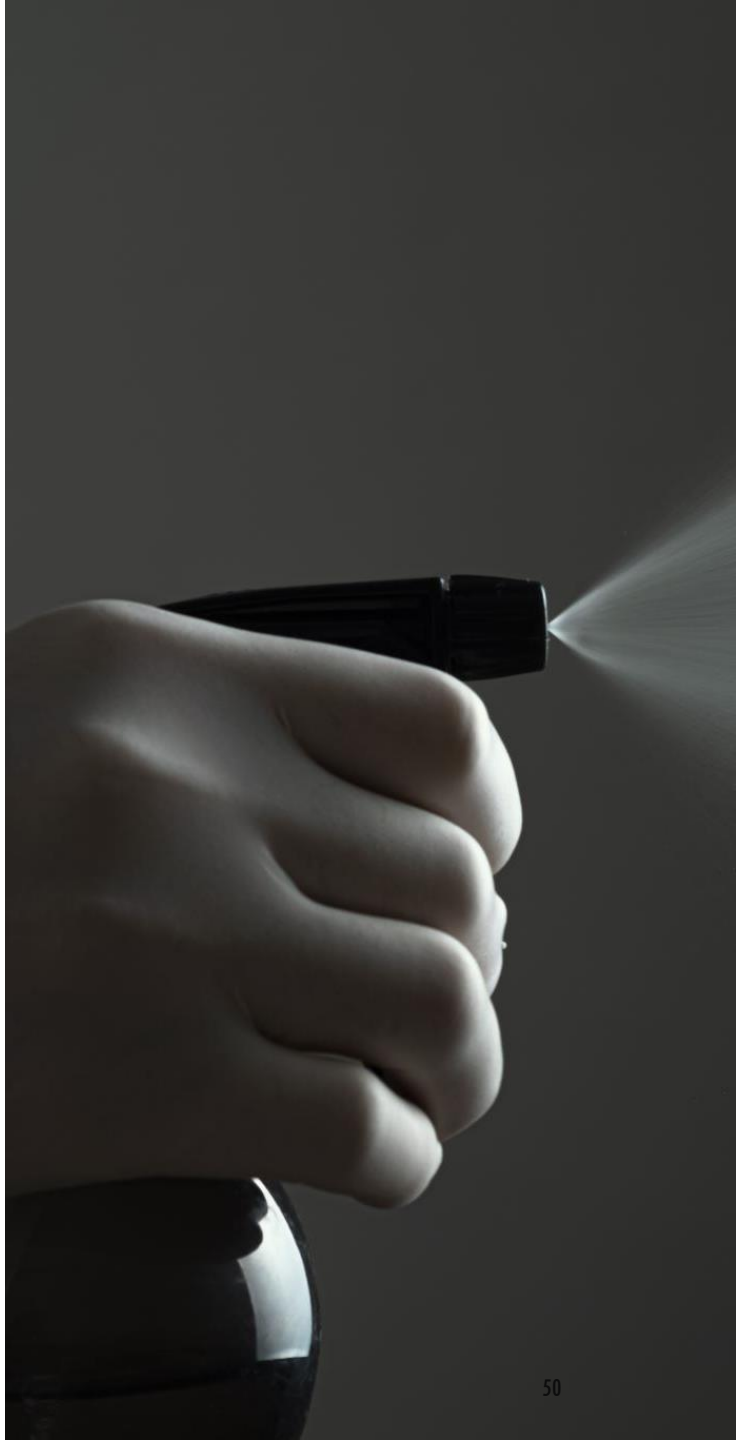
KEY CONCEPTS:

Stochastic Policy: Policies that output probabilities of actions rather than deterministic actions

Policy Gradient Theorem: Provides a way to compute the gradient of the expected reward with respect to the policy parameters

ACTOR-CRITIC METHODS

Definition: Actor-Critic methods combine value-based and policy-based approaches by using two models: the actor and the critic



KEY CONCEPTS:

Actor: Learns the policy that dictates the actions.

Critic: Evaluates the actions taken by the actor using a value function

ACTOR VS CRITIC

Actor: The actor is responsible for selecting actions based on the current policy. It updates the policy parameters in the direction that improves the expected reward. Essentially, the actor decides what action to take in a given state.

Critic: The critic evaluates the actions taken by the actor by estimating the value function. It provides feedback to the actor on how good or bad the action was, which helps in improving the policy.

IMPORTANT POINTS

Initialize the policy (actor) and value function (critic) with random weights.

The agent (actor) interacts with the environment by taking actions based on its current policy.

The environment responds with the next state and a reward.

The critic evaluates the action taken by the actor by estimating the value function (e.g., state-value or action-value).

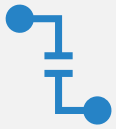
The critic computes the temporal difference (TD) error.

The actor updates its policy parameters in the direction suggested by the critic.

The critic updates its value function parameters to minimize the TD error.

Repeat steps 2-5 for a number of episodes or until convergence.

ADVANTAGES



Stability: By combining the actor and critic, the method benefits from the stability of value-based methods and the flexibility of policy-based methods.

Efficiency: The critic helps in reducing the variance of the policy gradient estimates, leading to more efficient learning.

RESOURCES

https://sefidian.com/2021/03/01/policy-g/

https://pylessons.com/A2C-reinforcement-learning

https://dilithjay.com/blog/actor-critic-methods

https://lilianweng.github.io/posts/2018-04-08-policy-gradient/

https://web.stanford.edu/~ashlearn/RLForFinanceBook/PolicyGradient.pdf

https://www.tutorialspoint.com/machine_learning/machine_learning_temporal_difference_learning.htm

https://en.wikipedia.org/wiki/Temporal_difference_learning

https://www.datacamp.com/tutorial/introduction-q-learning-beginner-tutorial

https://web.stanford.edu/class/cme241/lecture_slides/david_silver_slides/MDP.pdf

https://medium.com/@bibekchaudhary/markov-decision-process-mdp-simplified-1ae44cf53cc1